FLUTTER COURSE · SESSION 3 OF 15

Layouts & Styling

Last session you built a profile card with `Column`, `Row` and `Container`. Today you learn to place widgets anywhere on the screen, build lists that scroll, stack one widget on top of another, and style a whole app from one place.

BY THE END OF TODAY YOU CAN

Build any static screen by combining Padding, Row, Column, Expanded, ListView, GridView and Stack, and take every color and font from a single `ThemeData`.

How to use the project

The project has one small demo file for each concept, plus the Restaurant Menu App that uses all of them together. Each demo is its own tiny app.

In VS Code: open **Run and Debug** (`Cmd+Shift+D`, or `Ctrl+Shift+D` on Windows), pick an app from the dropdown under **Session 3**, and press **F5**. It runs in debug mode, so the Widget Inspector and breakpoints work.

From the terminal: pass the demo's file with `-t`. Plain `flutter run` starts the Restaurant Menu App.

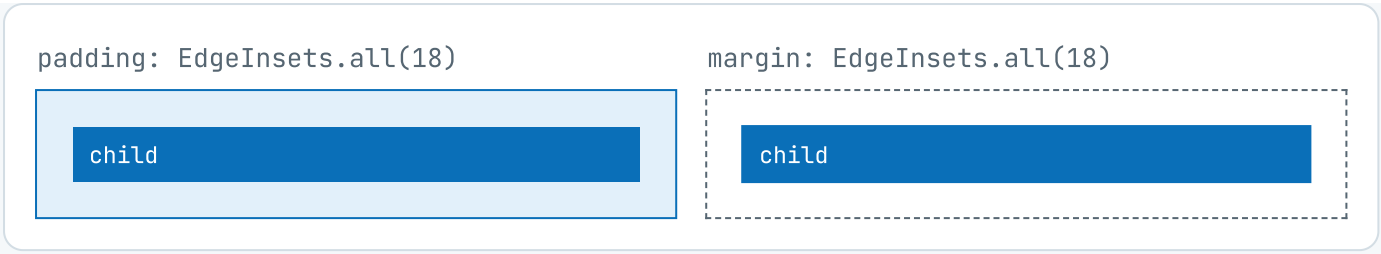
```
flutter run -t lib/day3_demos/demo1_padding_alignment.dart
```

While an app is running from the terminal, press `p` to turn on **Debug Paint**. It draws an outline around every widget, which is the fastest way to understand a layout. The diagrams on this page are drawn the same way.

The concepts, in the order we cover them

`demo1` Padding, margin, Alignment, Center

Padding is space inside a box, between its edge and its child. **Margin** is space outside a box, between it and its neighbours.



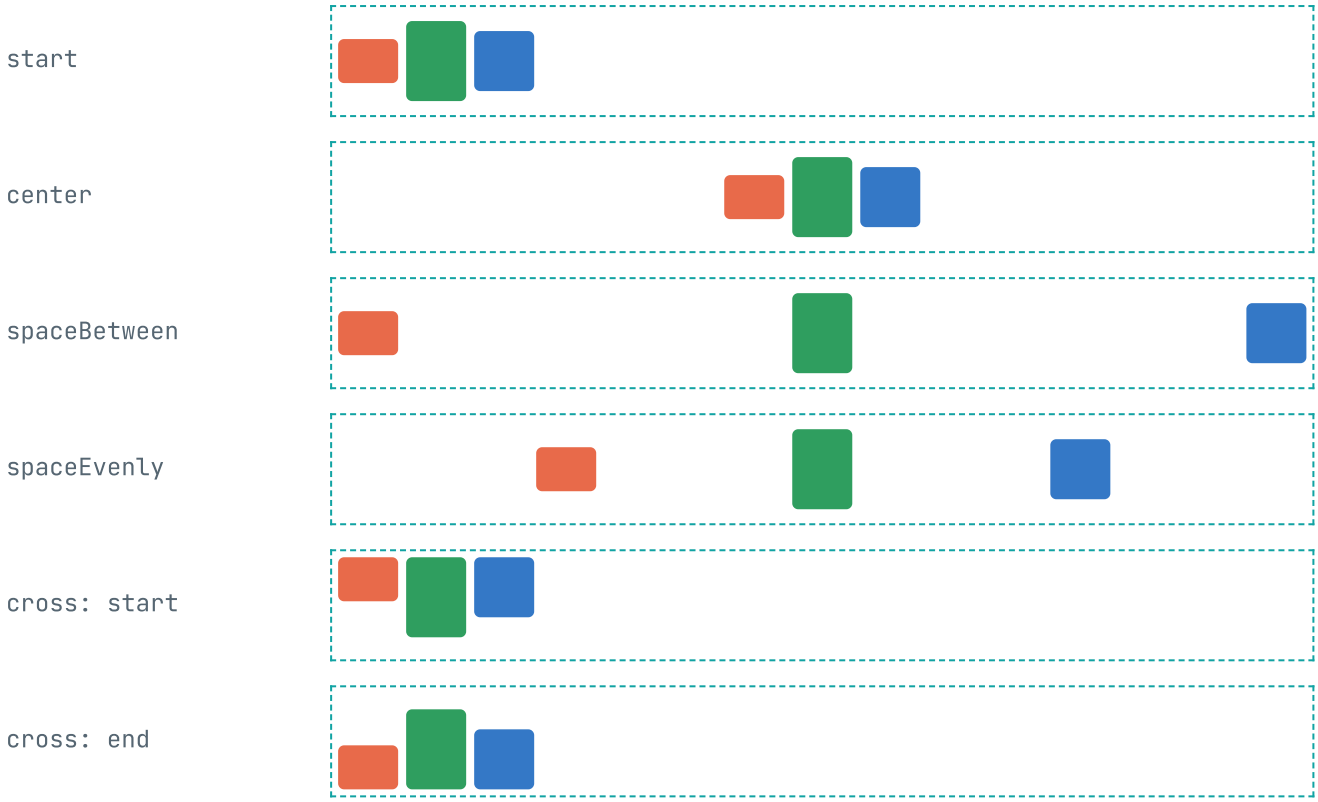

```
Container(
  margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
  padding: const EdgeInsets.all(12),
  alignment: Alignment.bottomRight, // where the child sits inside
  child: const Text('Hello'),
)
```

- `EdgeInsets.all`, `.symmetric` and `.only` cover most cases.
- `Alignment(x, y)` runs from −1 to 1. `(-1, -1)` is top-left and `(0, 0)` is the centre.
- `Center` is a shortcut for `Align(alignment: Alignment.center)`.

`demo2` MainAxisAlignment and CrossAxisAlignment

Every Row and Column has two directions. The **main axis** is the direction the children are laid out in. The **cross axis** runs across it.

WIDGET	MAIN AXIS	CROSS AXIS
`Row`	horizontal →	vertical ↓
`Column`	vertical ↓	horizontal →



The demo shows every value side by side. Change one, hot reload, and watch the boxes move.

---

demo3 Expanded and Flexible

A Row gives each child as much width as it asks for. When a long `Text` asks for more than the screen has, you get yellow-and-black overflow stripes. Wrapping it in `Expanded` tells it to use only the space that is left.

```
Row(children: [
  const Icon(Icons.check_circle),
  Expanded(            // takes all the leftover width
    child: Text('A very long sentence that would overflow…'),
  ),
])
```



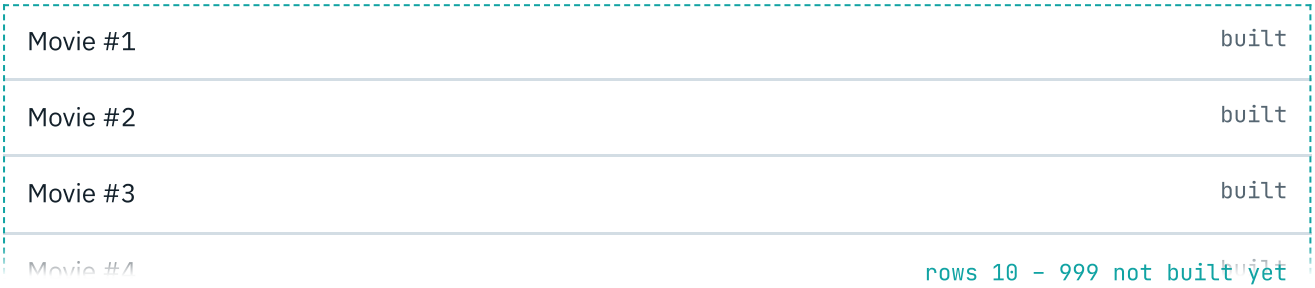
- **Expanded** must fill its share of the leftover space.
- **Flexible** may fill it, but can stay smaller if its child is small.

- `flex:` sets the ratio between siblings.
- Both only work as a direct child of a Row, Column or Flex.

---

demo4 ListView and ListView.builder

A Column does not scroll. Use a `ListView` when content can be taller than the screen.




```
ListView.builder(
  itemCount: movies.length,          // 1000 items
  itemBuilder: (context, index) {
    return ListTile(title: Text(movies[index]));
  },
)
```

- `ListView(children: [...])` builds every child at once. It's fine for a short, fixed list such as a settings page.
- `ListView.builder` builds only the rows that are on screen. Use it for long lists or lists that come from data. In the demo, watch the console: only about 10 of the 1000 rows are built.
- A ListView inside a Column must be wrapped in `Expanded` (or a `SizedBox` with a height) so it knows how tall it can be.

---

demo5 GridView

A GridView is a scrolling list laid out in columns.



```
GridView.builder(
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 3,         // 3 columns
    mainAxisSpacing: 8,
    crossAxisSpacing: 8,
    childAspectRatio: 1.5,     // width ÷ height of each tile
  ),
  itemCount: items.length,
  itemBuilder: (context, index) => ...,
)
```

- `GridView.count` is the short form for a few fixed items.
- Add `scrollDirection: Axis.horizontal` to scroll sideways. The category strip in the Restaurant app works this way.

---

demo6 Stack and Positioned

Row and Column place children next to each other. `Stack` places them on top of each other. The first child is at the back, and each later child is drawn over it.




```
Stack(children: [
  Image.asset('assets/images/dish1.jpg'),   // back layer
  Positioned(
    top: 10, left: 10,
    child: BestsellerBadge(),               // on top
  ),
])
```

Add `clipBehavior: Clip.none` to let a child hang outside the Stack, as in the notification dot on a cart icon in the demo.

`demo7` ThemeData, assets and custom fonts

Define colors and fonts once in `ThemeData`, then read them anywhere with `Theme.of(context)`. To restyle the whole app you change one file instead of hunting through every widget.

```
// main.dart: define once
MaterialApp(
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
    fontFamily: 'Poppins',
  ),
)

// any widget: read it
final colors = Theme.of(context).colorScheme;
Container(color: colors.primaryContainer)   // ✅ from the theme
Container(color: Color(0xFF009688))         // ❌ hardcoded
```

Images and fonts have to be registered in `pubspec.yaml` before Flutter can find them:

```
flutter:
  assets:
    - assets/images/
  fonts:
    - family: Poppins
      fonts:
        - asset: assets/fonts/Poppins-Regular.ttf
        - asset: assets/fonts/Poppins-Bold.ttf
          weight: 700
```

- YAML uses exactly 2 spaces per level. Tabs or wrong indentation break the file.
- After editing `pubspec.yaml`, run `flutter pub get` and do a **full restart**. Hot reload does not load new assets.
- The `family:` name in pubspec must match `fontFamily:` in ThemeData exactly.

The project: Restaurant Menu App

A menu for a restaurant called “Spice Route”. A row of category tiles sits at the top, and a scrolling list of dish cards fills the rest of the screen. Each card has a photo, a veg or non-veg mark, the name, the price in ₹ and a short description. Popular dishes carry a “Bestseller” badge. Tapping a category filters the list.

```
lib/
├─ main.dart · starts the app and applies the theme
├─ day3_demos/ · the 7 concept demos above
└─ restaurant/
   ├─ theme.dart · every color, font and card shape
   ├─ models/dish.dart · the Dish class
   ├─ data/menu_data.dart · categories and 8 dishes
   ├─ widgets/category_strip.dart
   ├─ widgets/dish_card.dart
   └─ screens/menu_screen.dart
assets/images/ · dish1.jpg … dish8.jpg
assets/fonts/ · Poppins Regular and Bold
```

Where each concept appears

FILE	WHAT TO LOOK FOR
`theme.dart`	`ColorScheme.fromSeed`, `fontFamily: 'Poppins'`, `cardTheme` for rounded cards, `appBarTheme`
`category_strip.dart`	A horizontal `GridView.builder` with one row of tiles. Colors come from `Theme.of(context)`.
`dish_card.dart`	`Stack` + `Positioned` for the Bestseller badge, `Expanded` so long names don't overflow, `Padding`, `Row` and `Column` for the text
`menu_screen.dart`	A `Column` holding the strip and a `ListView.builder` wrapped in `Expanded`
`pubspec.yaml`	The `assets:` and `fonts:` sections

Category tapping uses `setState`, which redraws the screen with the filtered list. We cover `setState` properly in Session 4. For now, just notice that it is there.

Errors you will see today

> `Yellow-and-black stripes: "A RenderFlex overflowed by 42 pixels"`
>
> A child is too wide for its Row (or too tall for its Column). Wrap it in `Expanded`, or add `overflow: TextOverflow.ellipsis` to the Text.

> `"Vertical viewport was given unbounded height"`
>
> You put a ListView or GridView directly inside a Column. Wrap it in `Expanded`.

> `"Unable to load asset: assets/images/…"`
>
> The path has a typo, the folder isn't listed in pubspec.yaml, or you used hot reload instead of a full restart.

> `The font doesn't change`
>
> The `family:` in pubspec.yaml doesn't match `fontFamily:` in ThemeData, or you haven't restarted.

> `"Incorrect use of ParentDataWidget"`
>
> You used `Expanded` or `Positioned` outside a Row/Column or Stack.

Practice task (in class)

Build a scrollable **Movies** screen with `ListView.builder`.

- [ ] Each card shows a poster image, the title, and a line with the year and rating.
- [ ] A long title doesn't overflow. Use `Expanded` and `TextOverflow.ellipsis`.
- [ ] No hardcoded colors. Every color comes from `Theme.of(context)`.
- [ ] At least one image is a local asset registered in pubspec.yaml.

Homework: extend the Restaurant app

- [ ] Add a "Spicy 🌶️" badge in the top-right corner of spicy dishes with a second `Positioned`. Add an `isSpicy` field to `Dish`.
- [ ] Add 4 more dishes with your own photos in `assets/images/`.
- [ ] Give the app a completely different look by editing only `theme.dart`.

- [ ] Show the number of dishes (“8 dishes”) below the category strip.
- [ ] Commit your work and push it to your GitHub repo.

Check yourself

<details>
<summary>What is the difference between Expanded and Flexible?</summary>

Expanded forces its child to fill its share of the leftover space. Flexible lets the child be smaller than that share if it doesn't need all of it.
</details>

<details>
<summary>Why use ListView.builder instead of ListView for 1000 items?</summary>

ListView.builder only builds the rows that are visible, so it stays fast. A plain ListView builds all 1000 at once.
</details>

<details>
<summary>In a Column, which direction does crossAxisAlignment control?</summary>

Horizontal. A Column's main axis is vertical, so its cross axis is horizontal.
</details>

<details>
<summary>In a Stack, which child is drawn on top?</summary>

The last one. The first child is at the back.
</details>

<details>
<summary>You added a new font file. What two steps make it work?</summary>

Register it under `fonts:` in pubspec.yaml and run `flutter pub get`, then do a full restart of the app. Also set `fontFamily` in ThemeData to the same family name.
</details>

NEXT SESSION

Session 4 · Navigation & Forms. Moving between screens, passing data, `setState` in depth, and forms with validation. You'll build a Contact Book app.